Assignment 4: Reliable Multi-Threaded File Transfer

Course: CSD2161/CSD2160 - Computer Networks

Date: March 16, 2026

Authors:

- **Elijah Chua** (e.chua@digipen.edu)
- **Guan Shao Jun** (shaojun.g@digipen.edu)

Brief: This report documents the design and implementation of a multi-threaded file transfer system. Key features include a **TaskQueue-based TCP server** for command routing and a custom **Selective Repeat UDP protocol**. The protocol is engineered for high reliability, utilizing a sliding window mechanism, individual packet retransmission timers (1000ms), and a **Selective Fast Retransmit** optimization to survive network instability and simulated packet loss.

Copyright (C) 2026 DigiPen Institute of Technology. *Reproduction or disclosure of this file or its contents without the prior written consent of DigiPen Institute of Technology is prohibited.*

Design Introduction

1. **Customized Reliable Communication Protocol**
   To achieve reliability over the UDP channel, our group decided to implement Selected Repeat sliding-window protocol. The server will be able to maintain a window of multiple unacknowledged packets in flight, up to 10 at a time. This maximizes the impact of network latency. The client can also handle out-of-order packets and writes them directly to the correct file offsets.

2. **Message format**
   All multi-byte field are converted to Network Byte Order(Big-Endian) using htonl() (4 bytes) / htons() (2 bytes) before transmission and reverted via ntohl()(4 bytes)/ntohs(2 bytes) upon receiving.

   UDP Message Format

UDP Data Datagram	Bytes
Session ID	4
File Length	4
File Offset	4
File Data Length	4
File Data	Variable (up to 1024)

3. **UDP Data Datagram**

Data	Bytes
Session ID	4
File Length	4
File Offset	4
File Data Length	4
File Data	Variable length (up to 1024 bytes)

UDP ACK Datagram

Data	Bytes

Flags	1
Ack Number	4
Sequence Number	4

TCP Message Format

REQ_DOWNLOAD

Data	Bytes
Command ID	1
Client's IP Address	4
Port Number	2
Filename Length	4
File Name	Variable length byte

RSP_DOWNLOAD

Data	Bytes
Command ID	1
Server's IP Address	4
Port Number	2
Session ID	4
File Length	4

RSP_LISTFILES

Data	Bytes
Command ID	1
Number of files	2
Length of File List	4
File Name Length	4
File Name	Variable length byte

REQ_QUIT, REQ_LISTFILES, DOWNLOAD_ERROR

Data	Bytes
Command ID	1

4. **ACK Scheme & Error Recovery**
   Our Selective Repeat protocol uses individual packet timers and out-of-order tracking to survive packet loss.

   Client Tracking: Client tracks successful data chunks using a std::set<uint32_t>. This helps to prevent writing duplicated packets to the disk if an ACK is lost. Received data is written directly to the correct position using std::ofstream::seekp.

   Server sliding window: Server tracks unacknowledged packs in a std::map<uint32_t, PackSlot>. If it continuously polls the socket using select() with a non-blocking 10ms timeout to listen for incoming ACKs.

   Selective Retransmission: In each loop, server checks the elapsed time for every unacknowledged packet in the window. If a specific packet has not been acknowledged within 1000ms (1second), only that packet is retransmitted.

   Retry Limit & Connection Safety: To prevent the server from looping indefinitely on a broken link, each packet maintains a retry counter. If any packet fails to be acknowledged after 20 attempts, the server aborts the transfer and closes the socket. Server experiences more than 1000 10ms timeouts (10 seconds total), it assumes client disconnected and aborts the thread. The client also utilizes SO_RCVTIMEO with a 5000ms threshold to safely terminate if the server drops offline.

   Selective Fast Retransmit: In addition to the normal 1000ms timer, we also implemented Fast Retransmit optimization. Server monitors out-of-order ACKs. If the oldest packet in the window remains unacknowledged while 4 newer packets are successfully ACKed, server assumes packet loss occurred and immediately retransmits rather than waiting for 1000ms time to expire.

5. **Transaction Sequence**
   TCP Handshake: Client sends REQ_DOWNLOAD containing filename and its target UDP port.

   Server Initialization: Server verifies file exists, generates a session ID and replies

with RSP_DOWNLOAD message containing file size and server's UDP port.

Multi Threading & Port Reuse: The server spawns a detached `UdpSendFileThread` for the download. We utilize the `SO_REUSEADDR` socket option, allowing multiple concurrent server threads to bind to the same server UDP port without crashing.

Dynamic ACK Routing: To ensure ACKs reach the correct thread, client dynamically extracts the exact return address from recvfrom's fromAddr parameter and sends ACK directly back to the active socket.

Progress Tracking: Client receives data, dynamically renders a visual percentage-based progress bar in the console.

6. **<u>Communication Performance</u>**
   By using Selective Repeat sliding window protocol with a pipeline of 1- concurrent packets, it effectively bypasses the Round-Trip Time and delivers fast downloading while ensuring packets are only resent if it is necessary.

Verification

1 to 1 transfer:

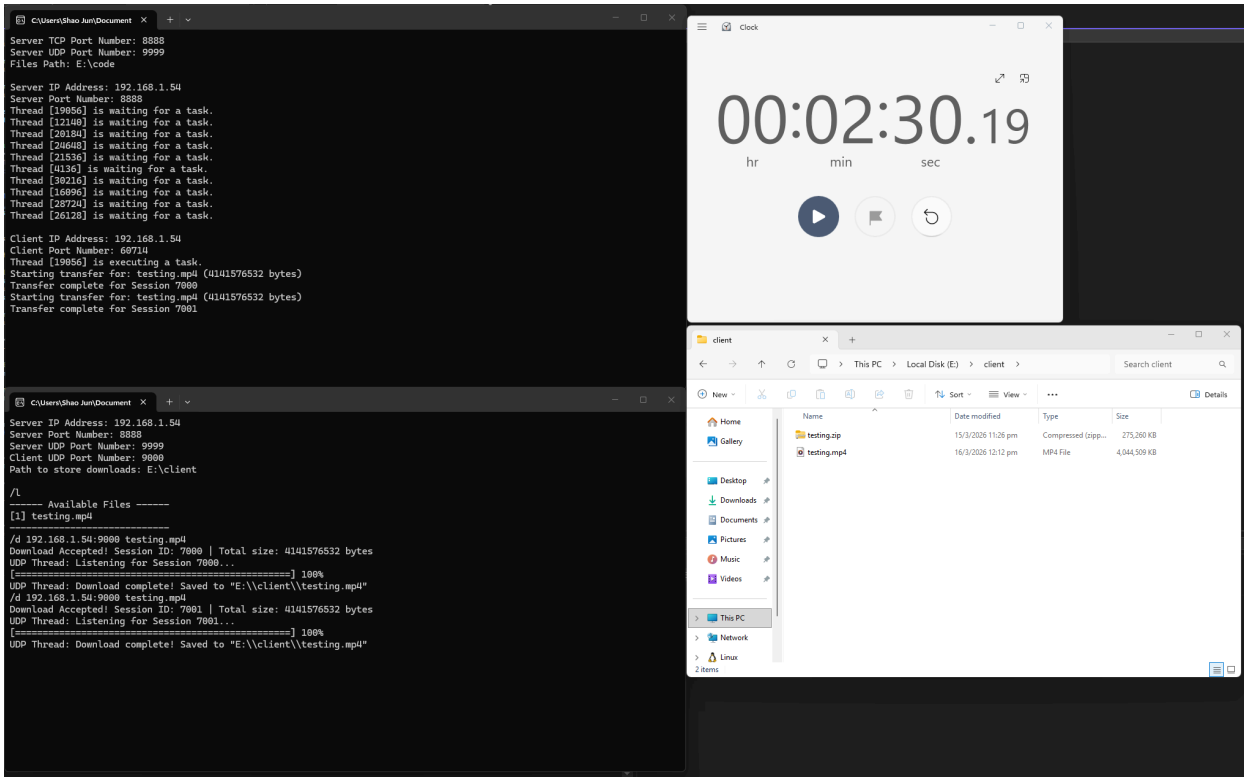
C:\Users\Shao Jun\Document

```
Server TCP Port Number: 8888
Server UDP Port Number: 9999
Files Path: E:\code

Server IP Address: 192.168.1.54
Server Port Number: 8888
Thread [29900] is waiting for a task.
Thread [9864] is waiting for a task.
Thread [24400] is waiting for a task.
Thread [11376] is waiting for a task.
Thread [29296] is waiting for a task.
Thread [18084] is waiting for a task.
Thread [16336] is waiting for a task.
Thread [22852] is waiting for a task.
Thread [8500] is waiting for a task.
Thread [10920] is waiting for a task.

Client IP Address: 192.168.1.54
Client Port Number: 56141
Thread [29900] is executing a task.
Starting transfer for: testing.zip (281865898 bytes)
Transfer complete for Session 7000
```

C:\Users\Shao Jun\Document

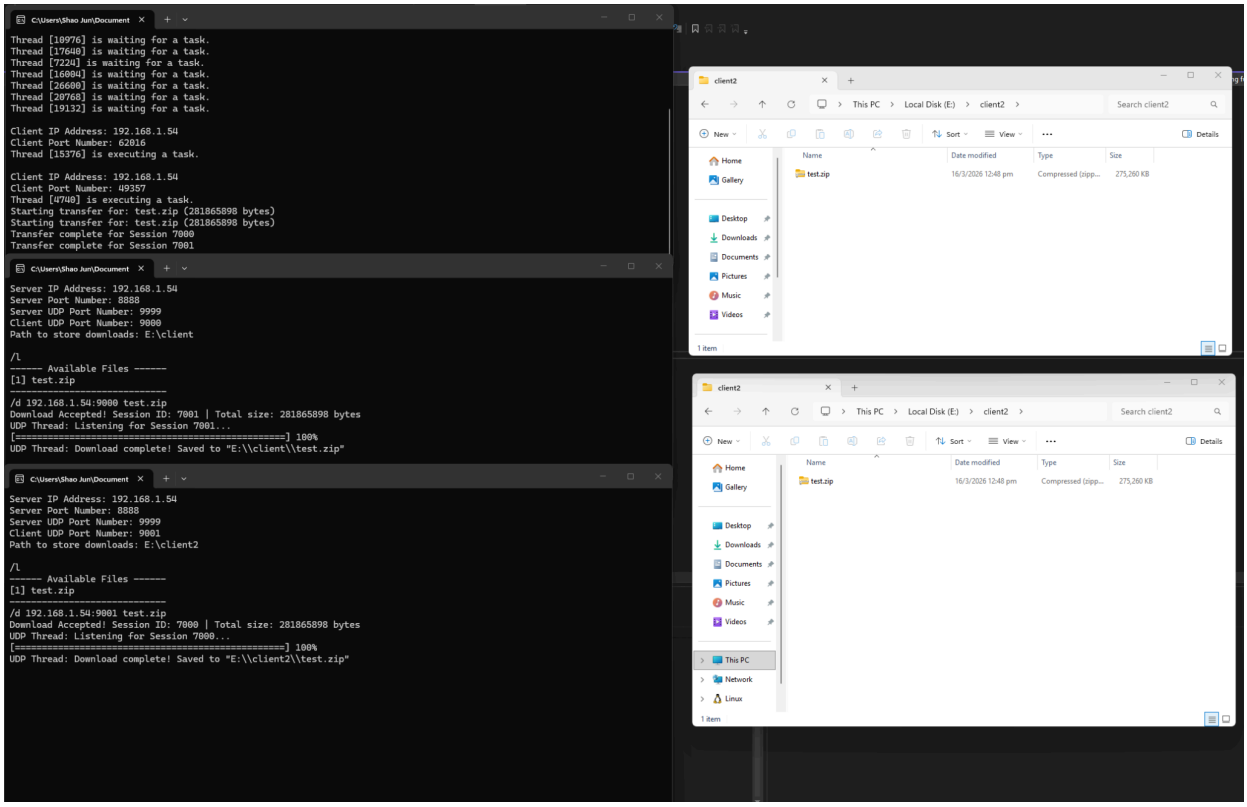
```
Server IP Address: 192.168.1.54
Server Port Number: 8888
Server UDP Port Number: 9999
Client UDP Port Number: 9000
Path to store downloads: E:\client

/l
------ Available Files ------
[1] FbR3JB0aQAEkHy2.jpg
[2] testing.zip
-----------------------------
/d 192.168.1.54:9000 testing.zip
Download Accepted! Session ID: 7000 | Total size: 281865898 bytes
UDP Thread: Listening for Session 7000...
[==================================================] 100%
UDP Thread: Download complete! Saved to "E:\\client\\testing.zip"
```

4GB Stress Test

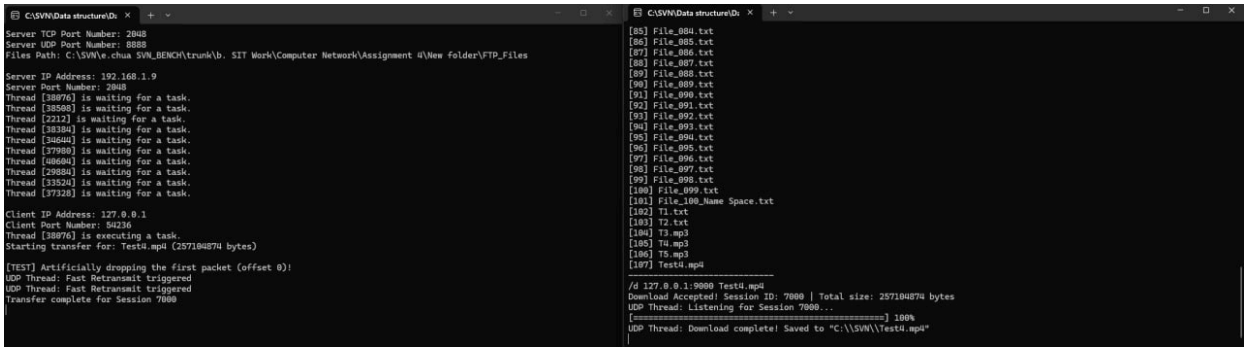


The sliding window protocol maintained a throughput of 27.6 MB/s, completing the transfer in approximately 2 minutes and 30 seconds without a single corrupted byte, verifying our out-of-order `std::set` tracking and `std::ofstream` seeking logic.

Multi-Client



We launched the server and connected multiple client instances simultaneously. We verified that the server could asynchronously handle `/l` (list files) commands from one client while actively transferring a file to another, proving that the `TaskQueue` thread pool and detached UDP sender threads functioned without blocking the main TCP listener.



We included a Selective Fast Retransmit optimization. When packet offset 0 was dropped, on purpose, the server detected the gap via subsequent ACKs and retransmitted immediately, rather than waiting for the full second timeout.

Individual Contribution

`Elijah Chua`: Implemented the multi-threaded server architecture utilizing the `TaskQueue` model, the server TCP routing logic, directory iteration with exception handling, and the Selective Repeat with fast transmission sliding window mechanism (`PacketSlot` map) for high-speed UDP transmission. Test the system as well.
`Guan Shao Jun`: Implemented the client-side network architecture, including the TCP command parser, manual byte-packing serialization to prevent compiler padding corruption, the UDP receiver thread, and the dynamic progress bar UI. Test the system as well.